

Data Analysis Handbook

How to Collect, Clean, Analyze, & Visualize Data

Angel Alcala Ruiz

2023-10-19

Table of contents

Preface	6
I PART 1. ABOUT DATA	7
1 What is Data Analysis?	8
1.1 Talking About Data	8
1.2 The Data Analysis Process	9
2 How to Think About Data	11
2.1 Introduction	11
2.2 Referring to Variables in Data	12
2.2.1 Qualitative vs Quantitative	12
2.2.2 Continuous vs Discrete	13
2.2.3 Scales of Measurement	13
2.2.4 Independent vs Dependent	15
3 Data Storage	16
3.1 Data Tables	16
3.2 Data Organization	17
4 How Programs Handle Data	18
II PART 2. PROGRAMMING IN R	19
5 Intro to R	20
6 R API's	21
III PART 3. PROGRAMMING IN PYTHON	22
7 Introduction to Python	23
7.1 Variables and Assignments	23
7.2 Loops in Python	23
7.3 Conditionals	23

7.4	Functions	23
7.5	Classes in Python	23
8	Python API's	25
8.1	Python API's	25
8.2	Working with Weather and City API's	26
8.3	API's and Geospatial Visualization	26
8.4	Further Reading and Helpful Links	27
IV	PART 4. SQL	28
9	SQL	29
9.1	Introduction to SQL	29
9.2	Advanced SQL Queries	30
9.3	Data Modeling	30
10	Advanced SQL	31
10.1	Introduction to SQLAlchemy	31
10.1.1	SQLAlchemy	32
10.1.2	Python Classes	33
10.1.3	SQLAlchemy's Object-Relational Mapper (ORM)	34
10.2	Advanced Usage of the SQLAlchemy ORM	34
10.3	Introduction to Flask and Serving Data with APIs	35
11	Data Collection	36
11.1	Scraping HTML	36
11.2	Web Scraping with CSS Selectors	37
11.3	Automated Browsing	37
12	NoSQL Databases	38
12.1	Introduction to NoSQL and MongoDB	38
12.2	PyMongo and Advanced Queries	39
12.3	Aggregation, Analysis, and Integration with MongoDB	39
13	Extract, Transform, and Load	41
13.1	Extract, Transform, and Load	41
13.2	Transformation and Cleaning with Regular Expressions	41
V	PART 5. DATA VISUALIZATIONS	43
14	Visualization Basics	44

15 Colors	45
16 Interactive Visualizations	46
16.1 Introduction to JavaScript Visualizations	46
16.1.1 Variables, Objects, Arrays	47
16.1.2 Conditional Statements and For-Loops	48
16.1.3 Functions	50
16.2 Functional Programing for Data Processing	50
16.2.1 The <code>map()</code> function	51
16.3 JavaScript with D3.js	53
17 Advanced Interactive Visualizations	54
17.1 Data Visualization with Leaflet	54
17.2 GeoJSON and Leaflet Plugins	55
17.3 Citi Bike Project with Leaflet	55
18 Tableau	57
18.1 Introduction to Tableau	57
18.2 Deeper into Tableau	58
VI PART 6. MACHINE LEARNING	59
19 Unsupervised Learning	60
19.1 Introduction to Machine Learning	60
19.2 Machine Learning in Practice	61
19.3 Principal Component Analysis (PCA)	61
20 Supervised Learning	63
20.1 Supervised Learning	63
20.2 Classification Models	64
21 Neural Networks and Deep Learning	65
21.1 Introduction to Advanced Machine Learning	65
21.2 Neural Network Models in the Real World	66
21.3 Machine Learning Decisions and Deployment	66
VII PART 8. BIG DATA	68
22 Big Data	69
22.1 Introduction to Big Data	69
22.2 Querying Big Data with PySpark	70
22.3 Optimizing Spark: Storage, Partitioning, and Caching	70

22.4 Databricks	71
References	72

Preface

The purpose of this book is to organize the notes from a couple of data analysis courses that I've taken. In a way, I want to merge both into one that can be a comprehensive guide to data analysis. This will be in an online book format that I believe can be beneficial for both learning and reviewing some of the material.

This book will cover various data analysis topics such data collection, data storage, cleaning, visualizations. The book will also cover useful tools such as SQL, R, Python, and a few others. As I continue to learn more about data analytics I would also be able to add more content to refine the book.

Part I

PART 1. ABOUT DATA

1 What is Data Analysis?

1.1 Talking About Data

When people talk about “data” what exactly do they mean? And what is data analysis? Who is a data analyst and what do they do? To help us to really complete our understanding of data, in the next few chapters, we will also discuss the following three notions of data:

- What are some of the things that analysts tend to think when discussing data?
- How do we store and organize data?
- How do programs handle data?

But first we can start by defining some of these terms:

Data In general, *data* are facts and statistics collected together for reference or analysis.

Data Analysis *Data analysis* is the collection, transformation, organization of data in order to draw conclusions, make predictions, and drive informed decision-making.

Data Analyst A *data analyst* is someone who collects, transforms, and organizes data in order to help make informed decisions.

Data Analytics *Data analytics* is the science of data.

Let’s take a look at the following example:

Example: *Luke Skywalker* grew up in *Tatooine*. He is a force sensitive *male*, *1.72 meters* tall, and uses a lightsaber.

There are numerous ways to store this data:

- Pen and paper
 - Photo
 - Audio
 - Word Document
 - Plain text file
-

In a computer, this data will be stored in a **file**.

File A *file* is simply a block of computer memory. It's a collection of related data typically organized as records.

- can be a few bytes to gigabytes in size
- a file format is a way of interpreting the bytes in a file

File Extension A *file extension* is the suffix that appears at the end of the file name and it indicates the type of file (i.e. the type of file).

- *.txt*, *.pdf*, *.docx*, *.mp3*, *.jpg* are all examples of file extensions.

1.2 The Data Analysis Process

Data analysis can be broken down into multiple parts and each plays a key role. These parts together define what's often referred to as the **data analysis process**.

1. **Identify** the question or problem
2. **Collect** the data
3. **Clean** the data
4. **Analyze** the data
5. **Interpret** the results

1.2.0.1 1. Identify

The data analysis process first begins with identifying the questions that we want to answer or the problems that we want to solve. In this step we want to ask smart and effective questions and also begin by putting things into context. It's also important to manage team and stakeholder expectations.

1.2.0.2 2. Collect

Once we've identified our questions and/or problems we can begin by collecting the data that will help us answer these questions. In this book we'll learn the tools that can be used to collect data and the best practices on how to store data.

1.2.0.3 3. Clean

In this step we clean the data which prepares it prior to analysis. Cleaning also allows for a much more efficient and organized analysis. Most data collected is dirty data, so in this book we will learn about tools in both Python and R on how to clean data.

1.2.0.4 4. Analyze

The analysis step is where we begin to answer our questions and solve our problems. Here, there are many methods that can be used for analysis such as mathematical methods, statistical methods, and visualizations that are used. This book will cover a few of these while also covering some very important preliminary concepts for visualizing data.

1.2.0.5 5. Interpret

The final step is to interpret and share our findings. Communication is highly important here because we have to be very considerate of who our audience is. This can be our team, our stakeholder's, the public, or even our family and friends. Each can require a different approach for sharing our results which presents a unique challenge from the other steps, but a highly important and highly rewarding one.

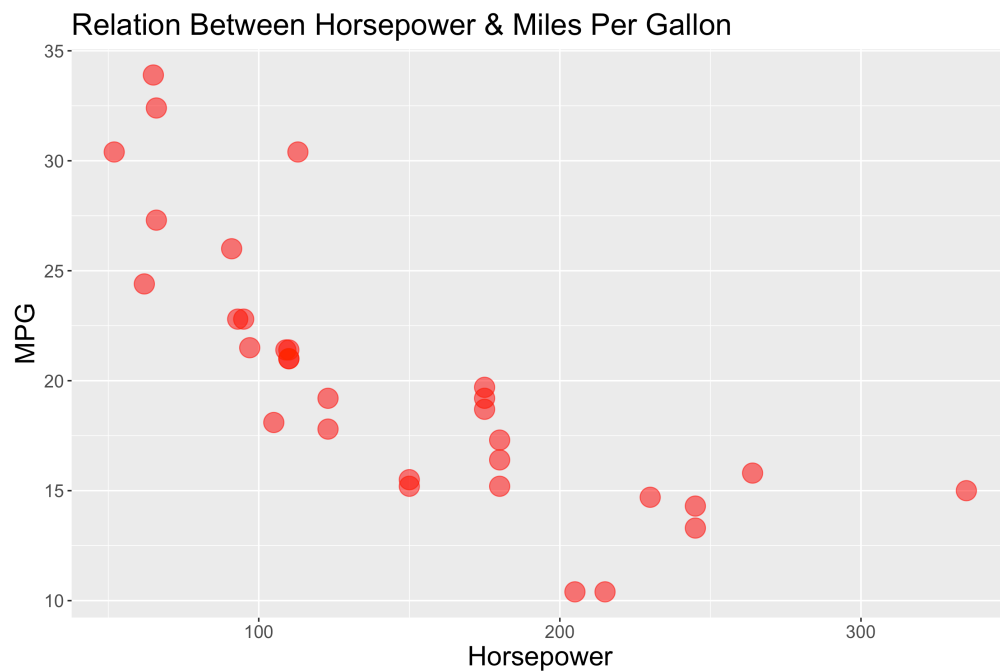
2 How to Think About Data

2.1 Introduction

A data analyst likely thinks of mathematical and statistical abstractions. This may involve putting things in terms of relationships or associations, perhaps theoretical mathematical models that take various forms; **linear**, **quadratic**, **non-parametric**.

Even before any analysis has begun when a data analyst thinks about data in general they will likely view that data in a table format with some visualization of that.

Example: Suppose we have some data about cars and we want to analyze their horsepower and miles per gallon. An analyst's view of this data should be some scatterplot of the following format



Here, each point represents a specific car with its horsepower and mpg. Even before any analysis begins, an analyst is likely viewing the relationship between horsepower and mpg in this way.

2.2 Referring to Variables in Data

In general, we should have the following in our minds when thinking of variables in our data

- qualitative (categorical) vs quantitative (numerical)
- continuous vs. discrete
- scales of measurement: nominal, ordinal, interval, ratio
- independent vs. dependent
- descriptors (predictors) vs. response
- input vs. output
- correlations
- theoretical models (linear, quadratic, etc.)

Let's define some of these terms

2.2.1 Qualitative vs Quantitative

Qualitative: Relating to, measuring, or measured by the quality of something rather than its quantity. This data is usually expressed as labels, groups, or categories. **Qualitative** data is also referred to as **categorical** data.

Qualitative data is grouped into two main types; nominal or ordinal.

Quantitative Relating to, measuring, or measured by the quantity of something rather than its quality. This data is expressed in numbers and can be measured. **Quantitative** data is also referred to as **numerical** data.

Quantitative data is grouped into two main types; **continuous** and **discrete**. The following are all examples of quantitative data

- weight in pounds
- length in centimeters
- dollar value of a company's stocks

2.2.2 Continuous vs Discrete

Continuous: Suppose f is a function that's defined on a set X of real numbers and $x_o \in X$. Then f is **continuous** at x_o if

$$\lim_{x \rightarrow x_o} f(x) = f(x_o).$$

Then we say that f is **continuous on the set X** if it is continuous at each number in X .

What this definition is saying is that a function has no discontinuous points such as a *jump* or a *break* along a given interval.

1. Jump discontinuity

$$\lim_{x \rightarrow x_o} f(x) \text{ does not exist}$$

2. Removable discontinuity

$$\lim_{x \rightarrow x_o} f(x) \neq f(x_o)$$

3. Infinite discontinuity

$$\lim_{x \rightarrow x_o} f(x) = \infty$$

Discrete A numerical type of data that refers to distinct elements. In other words, objects or elements that are unconnected. **Discrete** elements are almost always *finite*.

2.2.3 Scales of Measurement

There are 4 main types of scales of measurement with how variables are defined and quantified; nominal, ordinal, interval, ratio

Nominal A categorical type of data that represents a group or category that does not have any order or ranking. The following are examples of nominal data

- Hair color (black, brown, blonde, etc.)
- Vehicle types (SUV, coupe, sedan, crossovers, etc.)

Ordinal: A categorical type of data that represents a group or category that does have an order or ranking. The following are some examples of ordinal data

- Education level (high school, bachelor's, master's, etc.)
- Socioeconomic status (low, middle, upper)

Interval Interval data is a type of quantitative data that represents measurements with a consistent scale but without an absolute zero point. The differences between the values are meaningful, but the data lacks a meaningful absolute zero. That is, it doesn't make sense to say that something is "twice as much" as something else. The following are some examples interval data

- **Temperature:** the difference between 10°C and 25°C is the same as -5°C and 10°C . Notice that there's a lack of a true zero point because 0°C doesn't represent a lack of temperature.
- **Time of day:** there's also an equal interval between the times and on the 24 hour clock 00:00 does not represent a lack of time

Ratio Ratio data is a type of quantitative data that represents measurements with a consistent scale and it also contains an absolute zero point. This absolute zero point allows for meaningful calculations of proportions and ratios between measurements.

Ratio data allows for the most comprehensive statistical analysis when compared to the other types of data due to the ability to perform mathematical operations such as multiplication and division.

2.2.3.1 Table

Property:	Nominal	Ordinal	Interval	Ratio
Named	✓	✓	✓	✓
Equals (=)	✓	✓	✓	✓
Frequency	✓	✓	✓	✓
Mode	✓	✓	✓	✓
Order		✓	✓	✓
Greater/less than		✓	✓	✓
Median		✓	✓	✓
Equal interval			✓	✓
Add, Subtract			✓	✓
Arithmetic mean			✓	✓
Deviation			✓	✓

Property:	Nominal	Ordinal	Interval	Ratio
Absolute zero				✓
Multiply, divide				✓
Geometric mean				✓
Coefficient of variation				✓

2.2.4 Independent vs Dependent

Independent An **independent** variable is the factor in an experiment that the researcher intentionally manipulates or changes to observe its effect. It's considered the cause or reason for an observed effect. Any changes in the dependent variable are hypothesized to be directly caused by variations in the **independent** variable. Also called:

- Input variable
- Exposure variable
- Explanatory or Manipulated variable

Dependent The **dependent** variable is what the researcher measures to see if it changes as a result of the manipulation of the independent variable. It represents the outcome or effect in an experiment. Changes in the **dependent** variable are considered to be **dependent** on the variations in the independent variable. Also called:

- Output variable
- Outcome variable
- Response variable

Correlation **Correlation** is a statistical measure that describes the extent to which two variables are linearly related, that is they change together at a constant rate. Correlation is a common tool that is used to describe relationships without necessarily making a statement about cause and effect.

3 Data Storage

In this chapter, we describe some of the common ways in which data is stored, under what format, and with what structure.

3.0.0.1 Learning Objectives

- Get exposed to a handful of common file-formats for data tables.
- Explain the difference between field delimiter files, and fixed width files.
- List three of the most common types of delimiters for delimited-based files.
- Discuss at least three spreadsheet data organization recommendations
- Describe what a data dictionary is (“metadata”, “code book”)

3.1 Data Tables

Data is commonly displayed in tabular form. This tabular form means that there are rows and columns to display data.

In rows items are arranged from left to right.

In columns items are arranged from top to bottom.

Therefore tables generally have the following format

Generally speaking the columns in a table represent **variables** of the dataset while the rows contain the **observations** or **attributes** for each record or entity. When data is stored like this it allows for the following:

- each row typically represents a single complete record or entity
- rows allow us to compare different data points related to the same entity
- each column represents a specific characteristic, attribute, or data field
- columns allow for the analysis of a single data field across all records

The difference between rows and columns? For databases, a row usually describes the properties or fields of a single entity. A column will represent a field that is common to all entities, not just one.

	variables						
observations							

Most data in tabular form can be stored in **spreadsheets** (*.xls*, *x.lxs*), but it's best to store in a **text** file (*.txt*, *.csv*, etc). In these text files data is separated by *field delimiters*.

Field Delimiters A **field delimiter** is a specific character that's used to separate data values in a text file

3.2 Data Organization

4 How Programs Handle Data

Part II

PART 2. PROGRAMMING IN R

5 Intro to R

6 R API's

Part III

PART 3. PROGRAMMING IN PYTHON

7 Introduction to Python

In this chapter we'll cover some of the basics of Python. This will include using Python as a calculator, variables in Python, loops, conditionals, and functions. This will help form a basic understanding of the fundamentals of programming with Python's syntax

7.1 Variables and Assignments

7.2 Loops in Python

7.3 Conditionals

7.4 Functions

7.5 Classes in Python

A **Class** in Python acts as a “blueprint” for creating objects. They are extremely useful for the reusability and organization of code.

An **instance** is an object that belongs to a class.

Let's consider the following example

```
# Create a class Square
class Square():

    def __init__(self, length):
        self.length = length

    def area(self):
        return self.length*self.length

    def perimeter(self):
```

```
        return self.length*4
```

In this case we have created a class

```
square1 = Square(23)
square2 = Square(23.123)
```

```
square1.length
```

23

```
square1.perimiter()
```

92

```
square1.area()
```

529

```
2+2
```


8 Python API's

With the help of the Pandas library, you can now perform quantitative analysis and create compelling data visualizations. However, you will need special tools to programmatically obtain and parse real-world data. That's where APIs come in. An application programming interface (API) is a set of functions that applications use to automate the back-and-forth communication between computers.

This week, you will use APIs to obtain and parse data from sources such as OpenWeatherMap, the U.S. Census, OMDb, and more. You will also have an opportunity to test your skills by plotting DataFrames from the API data by using Matplotlib.

8.1 Python API's

8.1.0.1 Overview

Today's lesson focuses on API calls. It begins with a brief overview of APIs and JSON traversal. Then, once you understand the fundamental process, you'll learn how you can make API requests with the Requests library. Links to an external site., using the OMDb Links to an external site. and New York Times Links to an external site. APIs.

8.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Make get requests with the Request library.
- Convert JSON into a Python dictionary.
- Read and apply API documentation.
- Sign up for and use an API key.

8.2 Working with Weather and City API's

8.2.0.1 Overview

Today's lesson will expand your API querying capabilities by introducing you to a variety of new APIs. The class will begin with a short review activity, where you will traverse a JSON file using your knowledge of Python. Then, after you've warmed up, you will work with the [OpenWeatherMap API](#) to an external site., which provides developers with various meteorological data.

Later in the lesson, you will learn about exception handling, which can be helpful when databases and API requests have missing values. Finally, the lesson will conclude with a few activities to help you build familiarity with API usage and reading complex documentation.

8.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Create applications with your knowledge of Python and an API's documentation.
- Load JSON from API responses into a Pandas DataFrame.
- Use `try-except` blocks to handle errors.

8.3 API's and Geospatial Visualization

8.3.0.1 Overview

In this class, you will be introduced to the Geoapify API to obtain information about geographic areas and the GeoViews Python library to create maps in Jupyter Notebook. Using these new tools and data from the U.S. Census, you will create visualizations to capture the socioeconomic trend of [banking deserts](#) to an external site..

8.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Use the Geoapify API to obtain information about geographic areas.
- Use the Census API wrapper.
- Explain the concept of rate limits and the importance of creating test cases before running large scripts.

- Dissect new API documentation.

8.4 Further Reading and Helpful Links

This module will introduce you to all the information that you'll need to become employer-ready. But, you can become employer-competitive by diving more deeply into the topics discussed in class. Remember that the data field is constantly changing. And as a professional, you'll be expected to stay up to date on the latest developments and conversations.

If you're interested in learning more about a particular topic or need a refresher, use the following resources:

- [JSON](#)
- [OMDb API](#)
- [The New York Times API](#)
- [Open Weather Map API](#)
- [The World Bank API](#)
- [Geoapify API](#)

Part IV

PART 4. SQL

9 SQL

In this module, you'll learn how to do data modeling, engineering, and analysis by using Structured Query Language (SQL). Applying your knowledge of DataFrames and tabular data, you'll create entity relationship diagrams (ERDs), import data into a database, troubleshoot common errors, and create queries that use data to answer questions. Databases are used everywhere—in small and large businesses and even by individuals working on personal projects. And, SQL is one of the most widely used database query languages. Its ability to organize and query data, especially on a large scale, makes SQL knowledge a skill that's highly sought after in the workforce.

9.1 Introduction to SQL

9.1.0.1 Overview

This lesson will introduce you to one of the most popular programming languages for working with databases: SQL. SQL programmers are in high demand, so this language is important to learn. The lesson will begin with a brief overview of SQL. Then, you'll complete a series of exercises that consist of using SQL to create both tables and simple queries.

9.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Run Postgres and pgAdmin.
- Create a SQL database and its tables by using pgAdmin.
- Define SQL data types, primary keys, and unique values.
- Load the data from CSV files into a database and then query the data.
- Explain the four basic functions of persistent storage—create, read, update, delete (CRUD)—and apply them to a database.
- Combine the data from multiple tables by using JOIN clauses.

9.2 Advanced SQL Queries

9.2.0.1 Overview

This lesson will expand your knowledge of SQL. It will begin with a deeper dive into queries by covering aggregates, grouping, and ordering. Then, after a series of practical activities that involve those topics, the lesson will turn to creating subqueries, creating views, and combining both of them.

9.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Create aggregate queries.
- Create subqueries to further explore data.
- Create views and run subqueries off of them.

9.3 Data Modeling

9.3.0.1 Overview

Today's lesson will focus on data modeling and the best practices for designing a database. First, you'll learn how to normalize data and how the tables in a database are related. Later in the lesson, you'll learn how to create visualizations of databases by using ERDs.

9.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Apply data modeling techniques to database design.
- Normalize data.
- Identify data relationships.
- Create visual representations of a database by using ERDs.

10 Advanced SQL

In this module, you'll receive an introduction to three new tools: SQLite, SQLAlchemy, and Flask. You'll use these tools to build on your knowledge of SQL database structures and querying methods. You'll also write and run Python code in a Jupyter notebook and create graphs by using Python.

10.1 Introduction to SQLAlchemy

10.1.0.1 Overview

This lesson will combine what we learned last week about SQL with our favorite programming language: Python. Before getting started, you'll need to install SQLite. SQLite is a SQL dialect that shares much of its syntax with PostgreSQL, but it's entirely serverless. The lesson will first introduce you to SQLAlchemy, a Python library that's designed to work with SQL databases. You'll then complete an activity in which you'll analyze weather data by using SQLAlchemy. After another activity in which you'll create new DataFrames based on United States census data, your instructor will lead you through a crash course in object-oriented programming (OOP).

10.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Connect to a SQL database by using SQLAlchemy.
- Perform basic SQL queries by using `engine.execute()`.
- Create Python classes and objects.
- Perform create, read, update, delete (CRUD) operations on data in a SQL database by using the SQLAlchemy object-relational mapper (ORM).

10.1.1 SQL Alchemy

SQLAlchemy is a Python library that allows users to access and manage SQL databases. SQLAlchemy provides a powerful interface to interact with relational databases.

SQLAlchemy bridges the differences among various SQL dialects. A single script that uses SQLAlchemy can perform the same query across the different SQL dialects such as, *PostgreSQL*, *SQLite*, *MySQL*, etc.

To help us begin to connect to a database we will use the function `create_engine()` from `sqlalchemy` to connect to the database.

```
from sqlalchemy import create_engine, text

# Path to sqlite file
database_path = "../data/sql-files/Census_Data.sqlite"

# Create engine to talk to the database
engine = create_engine(f"sqlite:/// {database_path}")
```

The `engine` above creates a `Dialect` object that is tailored towards `SQLite` and `Pool` object that will establish a DBAPI connection. So far we have **not** established the connection to the database. The connection happens when we make a function call to `engine.connect()` or `engine.begin()`.

First, let's use `engine.connect()` to perform a SQL query. Inside of `engine.connect()` we can pass a SQL query in the form of a string

```
# Query first 5 records in the the Database
with engine.connect() as connection:
    # Append LIMIT directly to your SQL string
    query = text("SELECT * FROM Census_Data LIMIT 5")
    result = connection.execute(query)

    for row in result:
        print(row)
```

```
('HOUSTON, TX', 'HOUSTON', 'TX', 3061887, 1775897, 684416, 11586, 230549, 1368287, 54180, 38
('CHICAGO, IL', 'CHICAGO', 'IL', 2702091, 1318869, 843633, 7554, 161478, 785374, 32800, 3705
('BROOKLYN, NY', 'BROOKLYN', 'NY', 2595259, 1126111, 870465, 8744, 297890, 509243, 48934, 38
('LOS ANGELES, CA', 'LOS ANGELES', 'CA', 2426413, 1068202, 324842, 15949, 273829, 1292382, 6
('MIAMI, FL', 'MIAMI', 'FL', 1820704, 1361009, 363514, 2250, 33144, 1162711, 27137, 322521, 3
```


Now let's establish a connection to get our data and we can also use the `pandas` library to work with our data

```
import pandas as pd

query_data = text("SELECT * FROM Census_Data")

with engine.connect() as connection:

    df = pd.read_sql(query_data, connection)

df.head()
```

```
/Users/dejesusalcala/anaconda3/lib/python3.11/site-packages/IPython/core/formatters.py:406: I
return method()
```

	CityState	city	state	Population	White Population	Black Population	Na
0	HOUSTON, TX	HOUSTON	TX	3061887	1775897	684416	
1	CHICAGO, IL	CHICAGO	IL	2702091	1318869	843633	
2	BROOKLYN, NY	BROOKLYN	NY	2595259	1126111	870465	
3	LOS ANGELES, CA	LOS ANGELES	CA	2426413	1068202	324842	
4	MIAMI, FL	MIAMI	FL	1820704	1361009	363514	

A good way to think about these commands is to think of them in the following way

- `text(...)` → tells SQLAlchemy “this is a SQL statement”
- `engine.connect()` → opens a database connection
- `with` → makes sure that connection gets cleaned up properly

10.1.2 Python Classes

Another useful aspect of SQLAlchemy is that it can also update a SQL database using Python classes.

Classes are essentially blueprints for Python objects; they allow developers to create organized variables with keys, values, and methods on the fly.

For example we can have the following

```
from sqlalchemy.orm import declarative_base
from sqlalchemy import Column, Integer, String, Float
```

```

# Sets an object to use the default declarative base in SQLAlchemy
Base = declarative_base()

# Creates Classes which will serve as the anchor points for our tables
class Dog(Base):
    __tablename__ = "dog"
    id = Column(Integer, primary_key = True)
    name = Column(String(255))
    color = Column(String(255))
    age = Column(Integer)

class Cat(Base):
    __tablename__ = "cat"
    id = Column(Integer, primary_key = True)
    name = Column(String(255))
    color = Column(String(255))
    age = Column(Integer)

# Create a specific instance of the Dog and Cat classes
dog = Dog(name = "Charles", color = "brown", age = 4)
cat = Cat(name = "Lucy", color = "gray", age = 7)

print(f"The dog's name is {dog.name}. He's a {dog.color} dog and he's {dog.age} years old.")
print(f"The cat's name is {cat.name}. She's a {cat.color} cat and she's {cat.age} years old.")

```

The dog's name is Charles. He's a brown dog and he's 4 years old.
The cat's name is Lucy. She's a gray cat and she's 7 years old.

10.1.3 SQLAlchemy's Object-Relational Mapper (ORM)

10.2 Advanced Usage of the SQLAlchemy ORM

10.2.0.1 Overview

This lesson will introduce you to the finer details of working with the SQLAlchemy ORM. You'll learn how to create complex queries, update rows, perform joins, and use ORM methods to run queries.

10.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Create classes that model tables by using the SQLAlchemy ORM.
- Perform CRUD operations on databases by using the SQLAlchemy ORM.
- Reflect existing databases.
- Review the table names in a database by using SQLAlchemy inspection.
- Plot query results that are retrieved by using SQLAlchemy.
- Run a t-test to validate differences in means.

10.3 Introduction to Flask and Serving Data with APIs

10.3.0.1 Overview

In today's lesson, you'll learn the fundamentals of both the web and client-server architecture. This will include using Flask to create a database-backed server and to design and implement API endpoints.

10.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Create and run a server by using Flask.
- Define endpoints by using a Flask decorator.
- Extract query-variable path values from GET requests.
- Run database queries on behalf of the client by using variable paths.
- Return JSONified query results from API endpoints.

11 Data Collection

This week will focus on collecting data from websites via web scraping techniques. First, you'll learn about how websites are made by using HTML and CSS. You will build your own simple websites to make sure that you understand how HTML and CSS work together to design the content and style of web pages. Then, you'll use your knowledge of HTML and CSS, alongside the Python package BeautifulSoup, to extract specific information from HTML code.

Once you have developed your web scraping skills, you'll take them even further by scraping real websites. You'll use the automated browsing package Splinter to extract and store data from multiple pages of the same website. There's a lot to cover, so let's get started!

11.1 Scraping HTML

11.1.0.1 Overview

This lesson will introduce you to HTML, then show you how to apply this knowledge to scraping a website. In the first part of class, you'll go over the basic structure of how HTML and CSS build a webpage so that you will have the foundational knowledge you need in order to scrape a website. In the second part of class, you will complete your first web scraping project by using the Python package BeautifulSoup.

11.1.0.2 What You'll Learn

- Identify HTML components in a website.
- Create a basic HTML document.
- Scrape data from a website by using BeautifulSoup.
- Style HTML elements by using CSS.

11.2 Web Scraping with CSS Selectors

11.2.0.1 Overview

This lesson will build on the topics from the previous lesson, so it's important that you are comfortable with HTML, CSS, and BeautifulSoup. You will work on more advanced web scraping activities by using CSS selectors to identify elements to extract. In addition, you will use Chrome DevTools to explore elements within websites that you are targeting.

11.2.0.2 What You'll Learn

- Use CSS selectors to scrape targeted elements.
- Use Chrome DevTools to identify elements and their CSS selectors.

11.3 Automated Browsing

11.3.0.1 Overview

This lesson will introduce you to automated web browsing and scraping by using Splinter. By using Splinter alongside BeautifulSoup, you will be able to automate the scraping process and perform more advanced web scraping projects. Since you'll be scraping real websites, rather than saved HTML, you'll learn about the ethics and legality of web scraping. This will help you make good choices if you decide to pursue your own web scraping projects.

11.3.0.2 What You'll Learn

- Use Splinter to perform automated browser actions.
- Automate the web scraping process by using Splinter and BeautifulSoup.
- Organize scraped information into a Python data structure.

12 NoSQL Databases

In this module, you'll be introduced to the concept of the NoSQL database with MongoDB, the leading database management system for this language. You will then expand on your data skills to work with non-relational documents, utilizing the PyMongo library to interface with your Mongo databases using Python. Finally, you'll learn how to build aggregation pipelines when retrieving data, and integrate skills you have learned in previous classes, such as Pandas, APIs, and Matplotlib.

12.1 Introduction to NoSQL and MongoDB

12.1.0.1 Overview

In this lesson, you'll learn about NoSQL databases with a focus on MongoDB. MongoDB is a document database that has more flexibility than a structured database, like a SQL database, for storing data. It can handle both smaller, personal projects and larger-scale projects that a company might require. For this module, MongoDB is a better choice than SQL because you'll use data that is stored in JSON files, which is very similar to the Mongo BSON format.

The lesson will begin with an overview of NoSQL databases and how they differ from SQL databases. Then, your instructor will lead you through an activity about working with basic MongoDB queries using the Mongo shell. Once you feel comfortable creating elements in a Mongo database, you'll learn how to update and delete data. Finally, you'll learn how to import CSV and JSON files into a Mongo database from your Terminal.

12.1.0.2 What You'll Learn

- By the end of this lesson, you will be able to:
- Identify the key differences between SQL and NoSQL databases to aid decisions around what kind of database to use in different situations.
- Create and connect to local Mongo databases.
- Perform create, read, update, delete (CRUD) operations on Mongo documents by using the Mongo shell.

- Import data from CSV and JSON files into a local MongoDB database.

12.2 PyMongo and Advanced Queries

12.2.0.1 Overview

In this lesson, you'll learn how to perform advanced queries to retrieve documents from a Mongo database. The lesson will begin with a brief introduction to the PyMongo library, which developers can use to work with Mongo from Python. Then, you will practice importing data that you can use with the PyMongo library. The rest of the class will follow the structure of instructor demo, student activity, and review so that you'll learn different advanced techniques for retrieving data from a Mongo database.

12.2.0.2 What You'll Learn

- By the end of this lesson, you will be able to:
- Use the PyMongo library to interface with MongoDB and perform basic CRUD operations.
- Select specific fields when retrieving documents from MongoDB.
- Use comparison operators to find documents in MongoDB.
- Use sort and limit with PyMongo when retrieving documents from MongoDB.

12.3 Aggregation, Analysis, and Integration with MongoDB

12.3.0.1 Overview

In this lesson, you'll learn how to create aggregation pipelines to assist with performing analysis on documents in a Mongo database. The lesson will begin with a class warm-up activity where you will import the data you will use for the whole lesson. Then, you'll be introduced to aggregation functionality and aggregation pipelines with PyMongo. Finally, during the second half of the class, you will work in groups on a mini-project to bring together the Mongo skills you have developed and integrate them with APIs, Pandas, and Matplotlib.

12.3.0.2 What You'll Learn

- By the end of this lesson, you will be able to:
- Use aggregation and aggregation pipelines with MongoDB to analyze a subset of a MongoDB collection.
- Convert a MongoDB result to a Pandas DataFrame.
- Import data from an API to save to MongoDB.
- Use data from a Mongo database to plot charts with Matplotlib.

13 Extract, Transform, and Load

13.1 Extract, Transform, and Load

13.1.0.1 Overview

For today's lesson, you'll learn to use Python and Pandas methods and functions and to list comprehensions to extract, transform, and clean data. Then, you'll pair with a partner to start working on the ETL mini project.

13.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Extract data by using Python and Pandas.
- Transform and clean data by using Python and Pandas.
- Parse string data into a Python dictionary.
- Use list comprehensions to make code more readable.
- Use regular expressions to manipulate string data.

13.2 Transformation and Cleaning with Regular Expressions

13.2.0.1 Overview

For today's lesson, you'll learn how to use regular expressions to find patterns and extract data from text and string data.

Towards the end of class you'll pair with a partner to begin working on the ETL mini project. Seek help from your instructor, TAs, and fellow students when you need it.

13.2.0.2 What You'll Learn

By the end of this lesson, you will:

- Use wildcards, sets, and escape characters in regular expressions to extract information.
- Use special characters in regular expressions to extract information.
- Create and use capture groups in regular expressions to extract information.
- Continue working on the ETL mini project.

Part V

PART 5. DATA VISUALIZATIONS

14 Visualization Basics

15 Colors

16 Interactive Visualizations

In this module, you will use Plotly, a JavaScript data visualization library, to create data visualizations that are attractive, accessible, and interactive. Libraries like Plotly offer interactivity, which can help your audience better understand your data and draw the same conclusions that you did. Later, you'll use JavaScript to enhance the interactive features of your visualizations by building out buttons and dropdown menus. You'll also build on your JavaScript foundation to manipulate, parse, transform, and retrieve data from external sources like .csv files and APIs. Finally, you'll deploy a polished data visualization to the web, all using the power of JavaScript.

16.1 Introduction to JavaScript Visualizations

16.1.0.1 Overview

In this lesson, you will be introduced to the fundamentals of JavaScript by creating basic charts with the Plotly library. First, you will learn how to store data using variables, objects, and arrays in JavaScript. Then, after you understand the basic concepts in JavaScript, you'll create a corresponding chart in Plotly. Next, you'll learn how to control flow with iterations and conditionals by using JavaScript. You'll also use Plotly to create multiple traces and the data array object that generates our multiple trace charts. Finally, you'll learn how to use functions in JavaScript and apply that knowledge to an activity where you will use Plotly to create a barchart to visualize data about movie ratings.

16.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Describe JavaScript variables, arrays, data types, and statements.
- Implement basic JavaScript control flow (functions, loops, if/else statements).
- Create functions in JavaScript.
- Create, update, and iterate JavaScript objects.
- Create basic charts, including bar charts and line charts, by using Plotly.

- Use Plotly's layout object to customize the appearance of charts.
- Annotate charts with labels, text, and hover text.

16.1.1 Variables, Objects, Arrays

Javascript variables are assigned with the keyword `let`

```
%%javascript  
  
let name = "Homer Simpson"  
  
let isEmployed = true  
  
let age = 39
```

<IPython.core.display.Javascript object>

JavaScript has objects which act similar to Python dictionaries

```
%%javascript  
  
let Berkeley = {  
  state: "California"  
  square_miles: 10.5  
};  
  
let LosAngeles = {  
  state: "California",  
  square_miles: 502.7  
}  
  
let Denver = {  
  state: "Colorado",  
  square_miles: 155  
};  
  
let Miami = {  
  state: "Florida",  
  square_miles: 55.3  
};
```

```
let Phoenix = {  
  state: "Arizona",  
  square_miles: 517.9  
};
```

<IPython.core.display.Javascript object>

We can access the elements of these objects with either **bracket notation** or **dot notation**

```
%%javascript  
  
// console.log is a print statement in JavaScript  
// dot notation is  
  
console.log(city["state"])  
console.log(city.state)
```

<IPython.core.display.Javascript object>

Elements can be added to JavaScript objects in the following way

```
%%javascript  
  
Berkeley.population = 117145  
LosAngeles.population = 3849000000  
Denver.population = 711463
```

<IPython.core.display.Javascript object>

16.1.2 Conditional Statements and For-Loops

There are also if, else, and else if conditions Javascript.

```
%%javascript  
  
// 1. Simple conditional  
if (condition) {
```



```

    // execute this statement if true
}

// 2. if-else
if (condition) {
    // execute this statement if true
} else {
    // execute this statement if false
}

// 3. else-if
if (condition1) {
    // execute this statement if condition1 is true
} else if (condition2) {
    // execute this statement if condition1 is false and condition2 is true
} else {
    // execute this statement if condition1 and condition2 are false
}

// 4. logical operators
if (condition1 && condition2) {
    // execute this statement only if condition1 and condition2 are true
}

if (condition1 || condition2) {
    // execute this statement if either condition1 or condition2 is true
}

```

<IPython.core.display.Javascript object>

Note that the statements to be executed are wrapped in curly braces {}. This is different that in Python where indentation is critical. In Javascript it's not necessary to indent, but it does make the code much more readable.

A for-loop in Javascript will have the following structure

```

%%javascript

// Prototypical use case increments loop counter by one on each iteration

for (let i = 0; i < 10; i++) {

```

```

    console.log("Iteration #", i);
}

// Looping through an array

let x = [0, 1, 1, 2, 3, 5, 8];

for (let i = 0; i < x.length; j++) {
    console.log(x[j]);
}

```

<IPython.core.display.Javascript object>

16.1.3 Functions

Creating functions in JavaScript consists of using the keyword `function` and the name of your choice for the function

```

%%javascript

// Create a function that adds two numbers

function addition(a, b) {
    let sum = a + b
    return sum;
}

```

<IPython.core.display.Javascript object>

16.2 Functional Programing for Data Processing

16.2.0.1 Overview

In this lesson, you'll learn about JavaScript methods that will help you organize and prepare data for charts in Plotly. The class will begin with an introduction to functional programming methods in JavaScript through preprocessing data for a Plotly chart. Once you understand functional programming conceptually, you will complete two activities applying your knowledge of JavaScript to create charts using Plotly. Then the lesson will transition to the `.filter`

method. Again, you'll learn the concept in JavaScript first, then have an opportunity to put your new skills to work by creating a bar chart in Plotly. The lesson will wrap up with an activity in which you will sort, slice, and reverse an array of Greek god search results to build a horizontal bar chart with Plotly.

16.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Apply the `map()` method and filter to parse data.
- Create and use arrow functions to simplify code.
- Use the `filter()` and arrow functions to manipulate and filter datasets.
- Use ES6 JavaScript methods.

16.2.1 The `map()` function

The `map()` function can be extremely useful in Javascript. One of the main uses for this function is iterating over an array which can help us create new arrays.

Here are some attributes of `map()`:

- creates a new array from calling a function for every array element.
- does not execute the function for empty elements.
- does not change the original array.

The syntax for `map()` is the following

```
array.map(function(currentValue, index, arr), thisValue)
```

where

Parameter	Description
<code>function()</code>	Required. A function to be run for each array element.
<code>currentValue</code>	Required. The value of the current element.
<code>index</code>	Optional. The index of the current element.
<code>arr</code>	Optional. The array of the current element.
<code>thisValue</code>	Optional. Default value undefined. A value passed to the function to be used as its this value.

Let's consider the following example

```
%%javascript

// Create array of numbers
const numbers = [1, 2, 3, 4];

// Pass a function to map and return the output
const squares = numbers.map(x => x*x)

// Print output
console.log(squares);

// Output:
// [1, 4, 9, 16]
```

<IPython.core.display.Javascript object>

We can also rewrite it in the following way which can help with readability

```
%%javascript

// Create a function to use
const squareFunction = x => x*x;

// Call the function
const squareArray = numbers.map(squareFunction);

console.log(squareArray);

// Output:
// [1, 4, 9, 16]
```

<IPython.core.display.Javascript object>

16.3 JavaScript with D3.js

16.3.0.1 Overview

In this lesson, you will learn how to add custom interactivity to your web visualizations. First, you'll be introduced to basic document object model (DOM) selection, manipulation, and events using [D3.js](#) to an external site., an industry-standard data visualization library written in JavaScript. The lesson will begin with an overview of D3 and D3.json from your instructor. Then, the lesson will turn to D3 Select and append. After a brief overview from your instructor, you will complete an activity using D3 to add a new row of data to a table. Then, after a walkthrough and activity using the `this` keyword, class will finish with a demonstration and activity on how to create dropdown menus by using Plotly.

16.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Create charts by using data from API calls.
- Use D3 for basic document object model (DOM) manipulation and event handling.
- Apply the `this` keyword to reference elements within a function.
- Dynamically manipulate the DOM through events.
- Manipulate charts through dropdown events and click events.
- Use `Plotly.restyle()` to create dynamic charts.

17 Advanced Interactive Visualizations

Creating effective, interactive maps is a critical skill for anyone in the data field. Used across a wide variety of applications and fields, maps allow us to explore, understand and make decisions about our world. In this module, you'll build on your JavaScript skills and the D3 library to create interactive maps. You'll use GeoJSON, a type of JSON file specifically designed to host geographical information, to develop points, linestrings, and polygons. You'll also explore non-spatial attributes, or data that is independent of all geometric considerations, and packaged in the hierarchical structure of GeoJSON files.

17.1 Data Visualization with Leaflet

17.1.0.1 Overview

In this lesson, you'll learn to create data visualizations with maps. First, your instructor will lead a brief demonstration of creating a Leaflet map. Next, you'll have the opportunity to create your own map using Leaflet and complete a brief labeling exercise. Then the class will transition to a discussion of Scalable Vector Graphics (SVG) shapes to use as markers such as points, circles, polygons, polylines and rectangles. With these new skills under your belt, you'll complete two more advanced activities: a visualization of the world GDP per capita, and enhancement to your existing US Cities map to include populations for the states and cities. Finally, the class will end with an introduction to working with GeoJSON data and an activity in which you will plot the occurrences of earthquakes

17.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Explain the benefits of visualizing data with maps.
- Create maps and plot data with the Leaflet.js library.
- Parse data from the GeoJSON format to create map-based data visualizations.
- Describe the concepts of layers and layer controls and how they are applied to add interactivity to maps.

17.2 GeoJSON and Leaflet Plugins

17.2.0.1 Overview

In this lesson, we'll continue our discussion of GeoJSON and learn how to extend the functionality of Leaflet with third-party plugins. The class will begin with an activity mapping New York City neighborhoods. Then your instructor will introduce you to Leaflet plugins, which are third party libraries that integrate with Leaflet to provide additional features. To ensure you're familiar with these new tools, you'll complete an activity to create a heatmap of crime in San Francisco using the `leaflet.heat` plug in. Next, you'll complete two additional activities with the guidance of your instructor: First you will rodent clusters in New York City, then you'll work with a partner to complete a Choropleth map of Florida's school districts. The class will end with a group activity where you will create a map on your own and present it to the class.

17.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Produce heat maps, marker clusters, and choropleth maps using third-party Leaflet.js plugins.
- Research how to use additional third-party Leaflet.js plugins and JavaScript libraries.
- Differentiate between maps and map elements for visualizing different datasets.
- Create and deploy custom interactive dashboards.

17.3 Citi Bike Project with Leaflet

17.3.0.1 Overview

You will further develop your Leaflet skills in this lesson. To get started, your instructor will demonstrate how to use the Python HTTP server. Next, you'll use the Citi Bike API to create a Leaflet map displaying the locations of Citi Bike stations in New York. After you've created your map, you will deploy the Citi Bike project to GitHub Pages.

17.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Complete an in-class group project using Leaflet.js.
- Deploy data visualizations to GitHub Pages.
- Draft a project proposal in a team setting.

18 Tableau

In this module, you will learn all about Tableau, a powerful business intelligence application that allows users to quickly create in-depth visualizations. This class will be light on programming to give you plenty of time to gain hands-on experience working with Tableau. During each class, you'll complete a series of exercises to expand your knowledge. To begin, you'll learn the basics of working in Tableau: importing, loading, joining, and splitting data. Later in the week, you'll practice more advanced skills, including how to create groups, sets, and custom calculations. The lessons will build in complexity, culminating in a mini-project where you will work in pairs or groups of three to explore data and create visualizations in Tableau using at least two of your own data sources.

18.1 Introduction to Tableau

18.1.0.1 Overview

In today's lesson, you'll be introduced to Tableau and many of its built-in functions. To begin, your instructor will give a brief introduction to Tableau and confirm that everyone has correctly installed Tableau Public. Then, you'll complete a series of hands-on activities on how to load and explore data, create visualizations, and join and split data in Tableau. After a short break, you will complete two more activities in Tableau: In the first, you will create charts to compare candies to practice your sizing, coloring, and labeling skills. In the final activity, you will practice your storytelling skills by developing a visualization to illustrate the best-paid college major and the impact of attending graduate school on long-term pay.

18.1.0.2 What You'll Learn

- By the end of this lesson, you will be able to do the following using Tableau:
- Import data from various sources, including CSV files and Excel spreadsheets.
- Perform joins on multiple data sources.
- Create and customize basic visualizations.
- Create storyboards.

18.2 Deeper into Tableau

18.2.0.1 Overview

In today's lesson, you'll expand your Tableau knowledge to gain experience in more advanced tasks, including custom calculations, maps, and level-of-detail calculations. To get started, your instructor will begin with an overview of how to create groups and sets in Tableau. Then, you'll practice these new skills in an activity using movie rental data. Next, you will practice calculations by creating visualizations using a dataset on motor vehicle accidents in New York City. After a short break, you will complete three more activities in the second half of class: One on mapping, a second on creating a dashboard, and in the third and final activity of the class, you'll practice creating level of detail (LOD) calculations with sample data.

18.2.0.2 What You'll Learn

- By the end of this lesson, you will be able to do the following using Tableau:
- Create groups and sets.
- Create maps and use built-in U.S. Census data.
- Create custom calculations.
- Apply LOD calculations.
- Create dashboards.

Part VI

PART 6. MACHINE LEARNING

19 Unsupervised Learning

Welcome to your first module about machine learning! Machine learning is a type of artificial intelligence that uses algorithms and statistical models to make decisions or predictions about data. With today's computing power, machine learning is changing the world faster than ever before.

This module focuses on a machine learning concept known as unsupervised learning. With unsupervised learning, algorithms use test data to build models that categorize the relationships among data points. For example, when you review an item for purchase on a website, unsupervised learning algorithms might identify related items that people frequently purchase together.

For using unsupervised learning models, you'll get introduced to a powerful Python package named `scikit-learn`.

19.1 Introduction to Machine Learning

19.1.0.1 Overview

This lesson will introduce you to machine learning, especially to one approach that we use in this field called unsupervised learning. Often with unsupervised learning, the goal is to let the machine learning help us figure out what groups, patterns, and structures to use for our data. The machine learning can do all that on its own—without us telling it what those groups, patterns, and structures are. Instead, it learns from the data that we give it and figures them out by itself. In this lesson, we'll learn how to use an unsupervised learning algorithm named K-means to help cluster our data into groups.

19.1.0.2 What You'll Learn

- By the end of this lesson, you will be able to:
- Recognize the differences between supervised and unsupervised machine learning.
- Define clustering and how people use it in data analysis.
- Apply the K-means algorithm to identify the clusters in a dataset.

- Determine the optimal number of clusters for a dataset by using the elbow method.

19.2 Machine Learning in Practice

19.2.0.1 Overview

This lesson delves more deeply into machine learning by beginning with a recap of the previous lesson and then delving into normalizing, preprocessing, and segmenting data. The goal of the lesson is to enable you to refine your unsupervised learning algorithms and flexibly apply them to various applications.

19.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Segment data.
- Prepare data for complex algorithms.
- Explain the importance of preprocessing data for unsupervised learning.
- Transform categorical variables into a numerical representation using Pandas.
- Scale data by using the StandardScaler module from scikit-learn.

19.3 Principal Component Analysis (PCA)

19.3.0.1 Overview

With machine learning, accuracy isn't the only performance metric to consider. Clustering algorithms, such as K-means, often suffer from something known as the **curse of dimensionality**—or too many features than we can make sense of in a single model. However, we can often fix this and improve the efficiency of our clustering by applying a technique called principal component analysis (PCA).

19.3.0.2 What You'll Learn

- By the end of this lesson, you will be able to:
- Explain what PCA is and how to use it to reduce the dimensionality of data.
- Explain how PCA relates to K-means and other applications in machine learning.
- Use PCA to reduce the number of features in an unsupervised learning setting.

20 Supervised Learning

Supervised learning involves connecting a large amount of observable information to those outcomes that we're interested in predicting. By the end of this module, you'll have gained familiarity with supervised learning models for continuous outcomes, like linear regression. You'll also have competency in running and evaluating classification models for logistic regression. Overall, you'll be proficient in using supervised learning approaches to predict categorical conclusions and outcomes.

20.1 Supervised Learning

20.1.0.1 Overview

The first lesson will introduce you to linear regression and mainly cover two topics. The first is a supervised learning model for classification for the logistic regression model. The second is evaluating whether a classification model produces adequate results. In the process, you'll use the `scikit-learn` data science package, specifically to train and evaluate models and make them more efficient and effective at determining the probability of outcomes.

20.1.0.2 What You'll Learn

By the end of this lesson, you'll be able to:

- Model and fit several supervised learning classification models using `scikit-learn`.
- Conceptualize and build training and test datasets for supervised learning analysis.
- Define classification in the context of machine learning.
- Evaluate classification algorithms using a confusion matrix and classification report.

20.2 Classification Models

20.2.0.1 Overview

In this lesson, you'll explore supervised learning in-depth. You'll learn more about classification models like support vector machines (SVMs), decision trees, and k-nearest neighbors (KNN) and about ensemble learning techniques by using the random forest model.

20.2.0.2 What You'll Learn

By the end of this lesson, you'll be able to:

- Explain how SVMs work as a binary classifier.
- Explain how decision trees and random forest work as classifiers, and how they differ from each other.
- Explain how the KNN algorithm works as a classifier and how it differs from other classifiers.
- Apply fundamental classification algorithms, namely SVMs, random forest, decision trees, and KNN in machine learning models.

21 Neural Networks and Deep Learning

In this module, we'll explore and implement neural networks using the TensorFlow platform in Python. We'll discuss the background and history of computational neurons as well as current implementations of neural networks as they apply to deep learning and the major costs and benefits of different neural networks. We'll compare these costs to traditional machine learning classification and regression models. Throughout the module, we'll practice implementing neural networks and deep neural networks across a number of different datasets including image, natural language, and numerical datasets. We'll close out the module by learning how to store and retrieve trained models for more robust uses.

21.1 Introduction to Advanced Machine Learning

21.1.0.1 Overview

In this lesson, you'll be introduced to neural networks, a powerful class of machine learning algorithms that are an essential tool for machine learning engineers. Neural networks are collections of perceptron models, which are related to logistic regressions. The lesson will begin with a warm-up activity in which you will use the logistic regression to build a binary classification model, which is the precursor to neural networks. Then we will turn our attention to the basics of neural networks with a series of lectures and hands-on activities that will introduce you to the perceptron model, the TensorFlow Playground, and Google Colab. The class will conclude with a final activity which will give you hands-on experience building your own neural network.

21.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Compare and contrast traditional machine learning classification and regression models and neural network models.
- Describe the perceptron model and its components.
- Implement neural network models using TensorFlow.

21.2 Neural Network Models in the Real World

21.2.0.1 Overview

This lesson focuses on deep learning, an essential technique for advanced machine learning solutions including medical image analysis, self-driving cars, and fraud detection. The lesson will begin with a lecture from your instructor defining deep learning models. Then, after you understand the fundamentals, you'll build a deep learning classification model that can adequately predict the class from our moons dummy dataset. Next, we'll turn our attention to model optimization. After a demonstration from your instructor, you will complete an activity using KerasTuner to create a model that can adequately predict scikit-learn's `make_circles` dataset. In the final activity, you will preprocess a medical dataset and create a deep learning model that can predict whether a patient will be diagnosed with myopia.

21.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Implement deep neural network models using TensorFlow.
- Explain how different neural network structures change algorithm performance.
- Save trained TensorFlow models for later use.

21.3 Machine Learning Decisions and Deployment

21.3.0.1 Overview

This lesson begins with a recap of what you've learned about data analysis and machine learning. Then you'll work with teams to develop a strategy on how you can solve the great debate question that we asked in our very first class: "Which do Americans prefer: Italian or Mexican food?" This will lead to an activity in which you will work with a group to brainstorm how to create a machine learning model that predicts a Yelp user's average rating of Italian restaurants. The second half of the class will focus on how to use Amazon SageMaker, which is built to streamline the process of training and deploying models. After a brief introduction from your instructor, you will complete a series of activities creating, deploying, and then deleting a notebook instance in Amazon SageMaker.

21.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Evaluate the trade-offs between machine learning models.
- Select and build an appropriate machine learning model for a given dataset and business case.
- Design an appropriate machine learning pipeline.
- Create and deploy a machine learning pipeline.

Part VII

PART 8. BIG DATA

22 Big Data

This week we'll demystify the term “big data” and give you some hands-on experience working with it. We'll start by reviewing Hadoop and its ecosystem. Within this context, we'll cover MapReduce and how it has improved the process for handling big data. We'll then move on to PySpark, the leading technology for handling big data. After diving into some of the technologies, we'll learn about advanced features of PySpark and how to optimize query execution times with large datasets by storing data in parquet format and partitioning and caching data

22.1 Introduction to Big Data

22.1.0.1 Overview

For today's lesson, you'll learn how to identify parts of the Apache Hadoop ecosystem. Then, you'll learn how to write Python scripts that implement the Apache MapReduce programming model. Next, you'll learn the differences between the Apache Hadoop and Apache Spark environments. Finally, you'll learn how to create and filter DataFrames using PySpark.

22.1.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Identify the parts of the Hadoop ecosystem.
- Write a Python script that implements the MapReduce programming model.
- Identify the differences between the Hadoop and Spark environments.
- Create a DataFrame by using PySpark.
- Filter and order a DataFrame by using Spark.

22.2 Querying Big Data with PySpark

22.2.0.1 Overview

For today's lesson, you'll learn how to group, aggregate, and order data, and learn how to parse, format a date to a timestamp, and plot time series data. Then, you'll learn how to create temporary views to run Spark SQL queries faster with large datasets. Finally, you'll combine PySpark and Spark SQL to run SQL queries.

22.2.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Apply grouping and aggregation functions to a dataset by using Spark.
- Parse and format date timestamps by using Spark.
- Use temporary tables to prepare data for SQL.
- Combine PySpark and SQL to run queries.

22.3 Optimizing Spark: Storage, Partitioning, and Caching

22.3.0.1 Overview

For today's lesson, you'll learn how to store data in parquet format and partition the parquet data to optimize query execution times. Then you'll practice caching data and determine optimal query execution times between partitioned and cached data.

22.3.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Compare the file storage types (other than tabular) that work the best for Spark.
- Understand how partitioning affects Spark performance.
- Explain the cause of shuffling and limit it when possible.
- Identify when caching is the best option.
- Explain how to broadcast a lookup table, and force it when it doesn't happen automatically.

- Set the shuffle partitions to an appropriate value and demonstrate how to cache data.

22.4 Databricks

22.4.0.1 Overview

In today's class, you will use Spark on Databricks to perform data analysis in the cloud. Through a series of exercises, you will gain hands-on experience with the Python and SQL interfaces of Databricks, with an emphasis on using the SQL interface for increasingly complex queries. The class will conclude with a group activity in which you will query a database in SQL, create a brief report with recommendations, and report your findings to the class.

22.4.0.2 What You'll Learn

By the end of this lesson, you will be able to:

- Explain the purpose, key features, and applications of Databricks.
- Set up a Databricks environment.
- Identify the key components of a Databricks environment.
- Navigate the Databricks workspace using `dbutils`.
- Import data into a new notebook by using Parquet files, CSV files, and S3.
- Explain the advantage of Parquet as a big data storage format.
- Perform complex data analysis, including joins, using the Python and SQL interfaces.
- Describe two advantages of using Databricks over PySpark for data analysis.

References